AWS DEVOPS PROJECT

Highly Available & Resilient Multi-AZ Web Infrastructure on AWS

PROJECT DOCUMENTATION

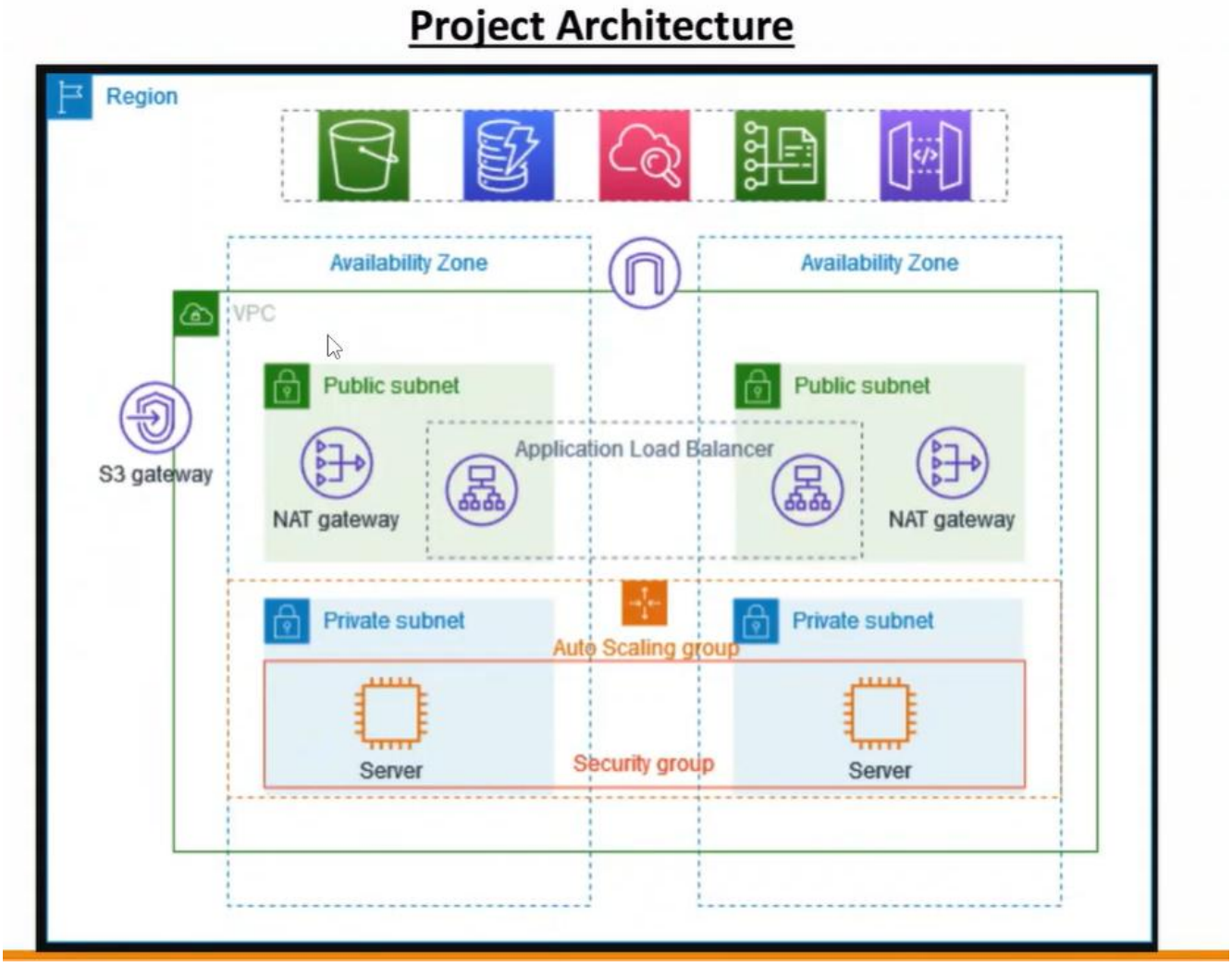


Figure: Project Architecture Diagram.

Document Contents

1. Project Overview

2. Objectives and Scope

3. Architecture Overview

4. AWS Services Used

5. Network Addressing Plan

6. Implementation Procedure

  6.1 VPC

  6.2 Subnets

  6.3 Internet Gateway

  6.4 Public Route Table

  6.5 Private Route Tables

  6.6 NAT Gateways

  6.7 Security Groups

  6.8 S3 Bucket

  6.9 S3 Gateway Endpoint

  6.10 EC2 Launch Template

  6.11 Target Group

  6.12 Application Load Balancer

  6.13 Auto Scaling Group

7. Application Request Flow

8. Validation and Testing

9. Security Considerations

10. Cost and Cleanup

11. Future Improvements

12. Conclusion

Appendix A. Suggested GitHub Structure

1. Project Overview

This project demonstrates the design and deployment of a web application architecture on Amazon Web Services (AWS), designed for high availability and horizontal scaling. The infrastructure was created in the Mumbai Region (ap-south-1) inside a custom Virtual Private Cloud (VPC) spanning two Availability Zones. Two public subnets support internet-facing networking components, while two private subnets host application EC2 instances.

An internet-facing Application Load Balancer (ALB) receives HTTP requests and forwards them to a target group. An Auto Scaling Group manages the EC2 instances launched from a reusable Launch Template. Public NAT Gateways provide outbound internet access for private instances. An S3 bucket and an S3 Gateway VPC Endpoint were also configured.

A basic Nginx web server was configured using EC2 User Data. The user reported that the application page was successfully displayed through the ALB DNS name.



Figure: Application running through ALB DNS.

2. Objectives and Scope

2.1 Objectives

- Create a custom VPC with an explicit IPv4 CIDR range.
- Separate public-facing components and private application servers using subnets.
- Configure Internet Gateway and route tables for public subnet routing.
- Configure NAT Gateways and private routes for outbound internet access.
- Restrict EC2 HTTP access to traffic from the ALB Security Group.
- Deploy Nginx on EC2 instances through Launch Template User Data.
- Route application requests through an Application Load Balancer and target group.

- Configure an Auto Scaling Group across two private subnets.
- Configure an S3 bucket and S3 Gateway Endpoint.

2.2 Scope

The implemented scope includes AWS networking, security groups, EC2 provisioning, HTTP load balancing, Auto Scaling configuration, and S3 endpoint connectivity. HTTPS/TLS, a custom domain, CI/CD, Infrastructure as Code, and a production application backend are not included unless separately implemented and tested.

3. Architecture Overview

The design uses one VPC across two Availability Zones. Each AZ contains one public subnet and one private subnet. The ALB is internet-facing and enabled in both public subnets. Each private subnet routes general outbound IPv4 traffic through a NAT Gateway in its corresponding AZ. The S3 Gateway Endpoint is associated with the two private route tables.

Application request flow: Client browser → Internet-facing ALB (HTTP/80) → Target Group → Nginx on private EC2 instances (HTTP/80).

Private outbound flow: Private EC2 → NAT Gateway in the same AZ → Internet Gateway → Internet.

S3 access flow: Private EC2 → S3 Gateway Endpoint → Amazon S3.

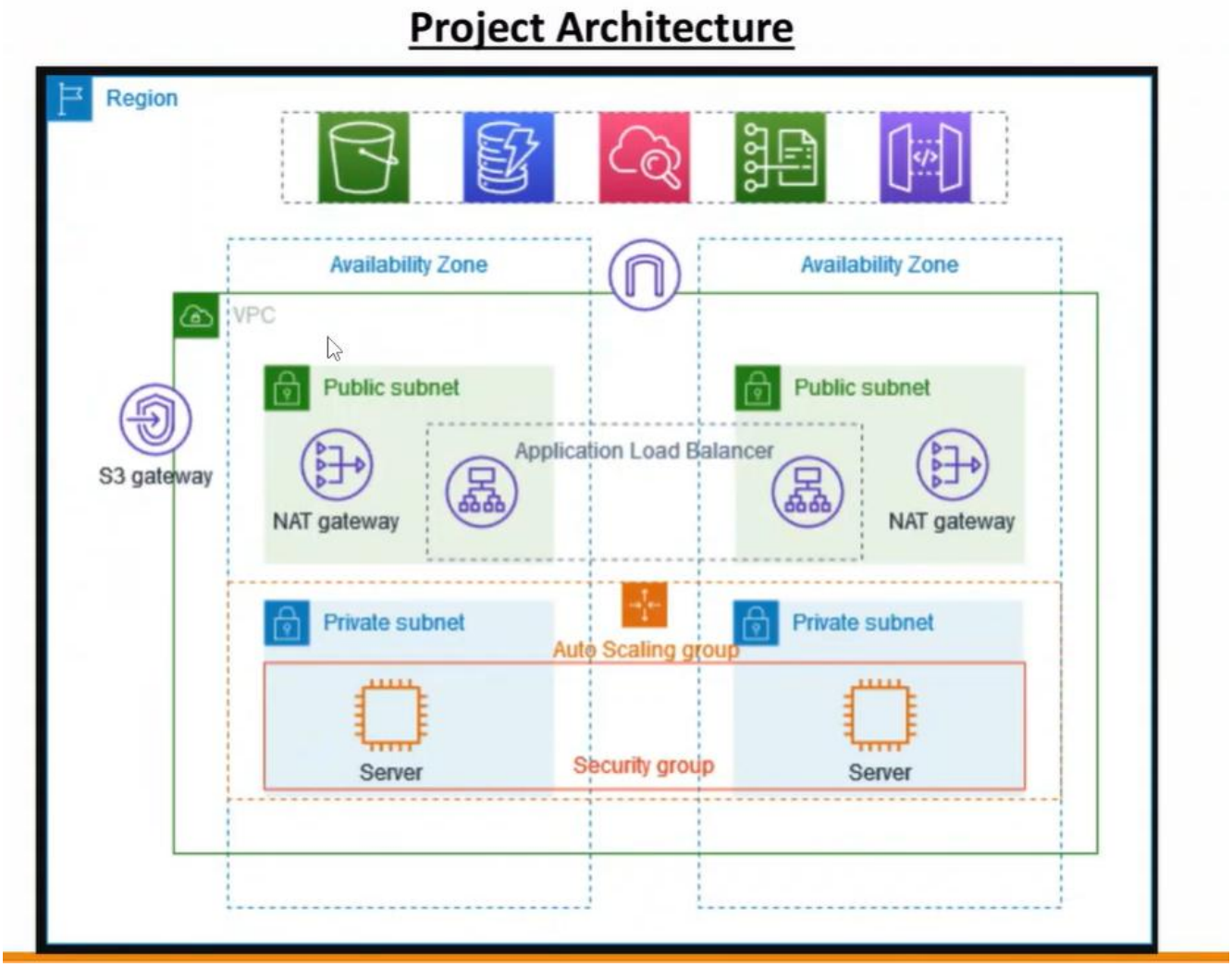


Figure: Architecture diagram.

4. AWS Services Used

Service / Component	Purpose
AWS Region (ap-south-1)	Hosts project resources in the Mumbai Region.
Amazon VPC	Provides an isolated network for project resources.
Availability Zones	Distributes network and application resources across two AZs.
Subnets	Separates public-facing networking components from private application instances.
Internet Gateway	Provides an internet gateway route target for public subnet traffic.
Route Tables	Directs VPC-local, internet-bound, NAT-bound, and S3-bound traffic.
NAT Gateway	Provides outbound internet connectivity for private instances.
Security Groups	Controls inbound and outbound traffic for ALB and EC2.
Amazon EC2	Runs the Nginx web server.
Launch Template	Defines AMI, instance type, security group, storage, and User Data.
Target Group	Registers application instances and performs HTTP health checks.
Application Load Balancer	Receives HTTP requests and forwards them to targets.
Auto Scaling Group	Maintains configured EC2 capacity and can scale when policies are configured.
Amazon S3	Provides object storage for files and assets.
S3 Gateway Endpoint	Routes S3 traffic without using NAT Gateway for that traffic.

5. Network Addressing Plan

Resource	Name	CIDR / Region
VPC	devops-project-vpc	10.0.0.0/16
Public Subnet AZ1	public-subnet-az1	10.0.1.0/24
Public Subnet AZ2	public-subnet-az2	10.0.2.0/24
Private Subnet AZ1	private-subnet-az1	10.0.11.0/24
Private Subnet AZ2	private-subnet-az2	10.0.12.0/24
Region	Mumbai	ap-south-1
Availability Zones	Two AZs	ap-south-1a and ap-south-1b as selected in console

Note: Availability Zone letter mappings can vary between AWS accounts. The actual subnet details shown in the AWS Console are the source of truth.

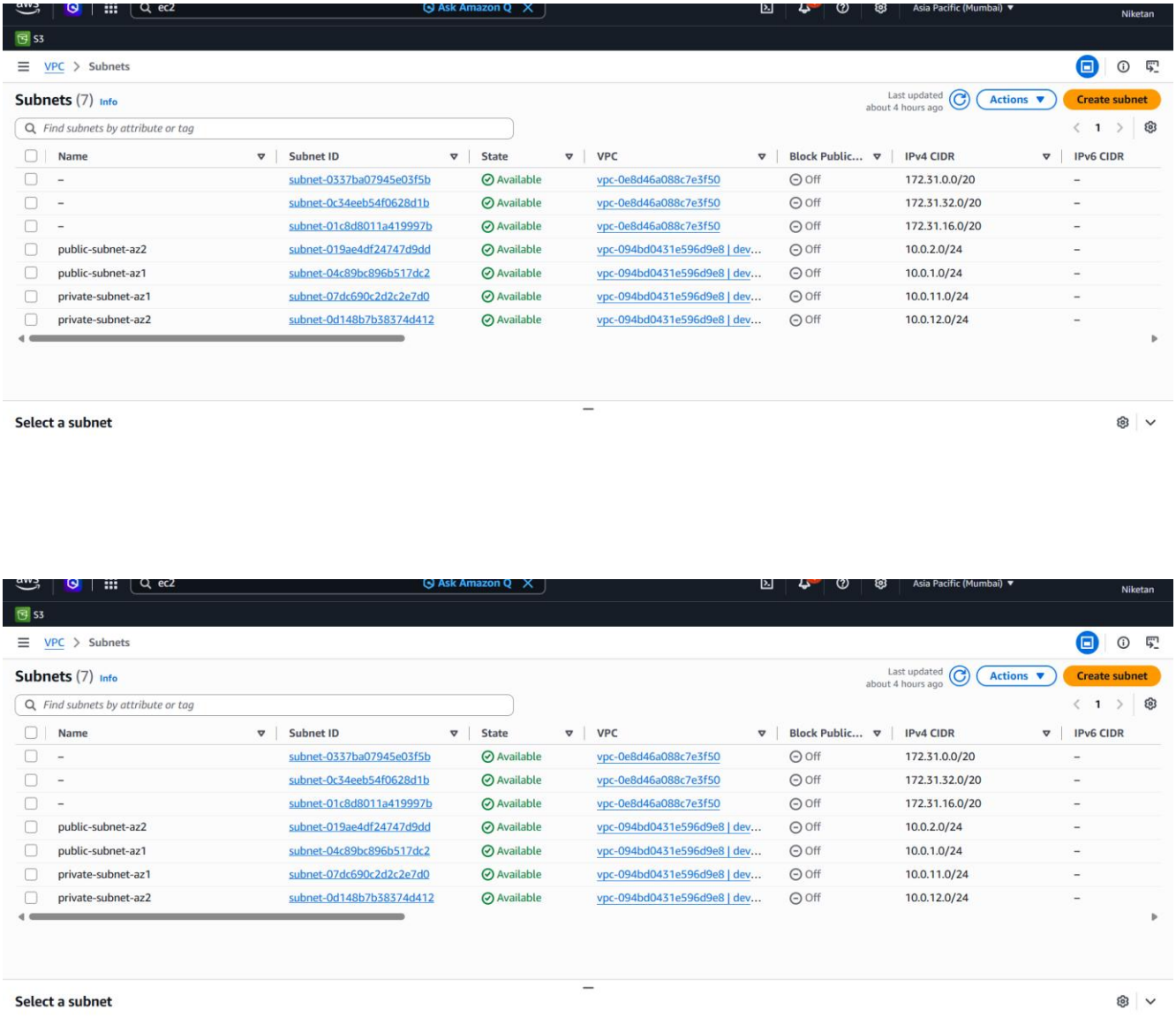


Figure: Four subnets with CIDR and Availability Zone.

6. Implementation Procedure

6.1 Create the VPC

Purpose: Create the isolated network boundary for the project.

1. Open the AWS Management Console and select the Mumbai Region (ap-south-1).
2. Open VPC → Your VPCs and choose Create VPC.
3. Select VPC only.
4. Set the name to devops-project-vpc and IPv4 CIDR to 10.0.0.0/16.
5. Create the VPC and verify that its state is Available.

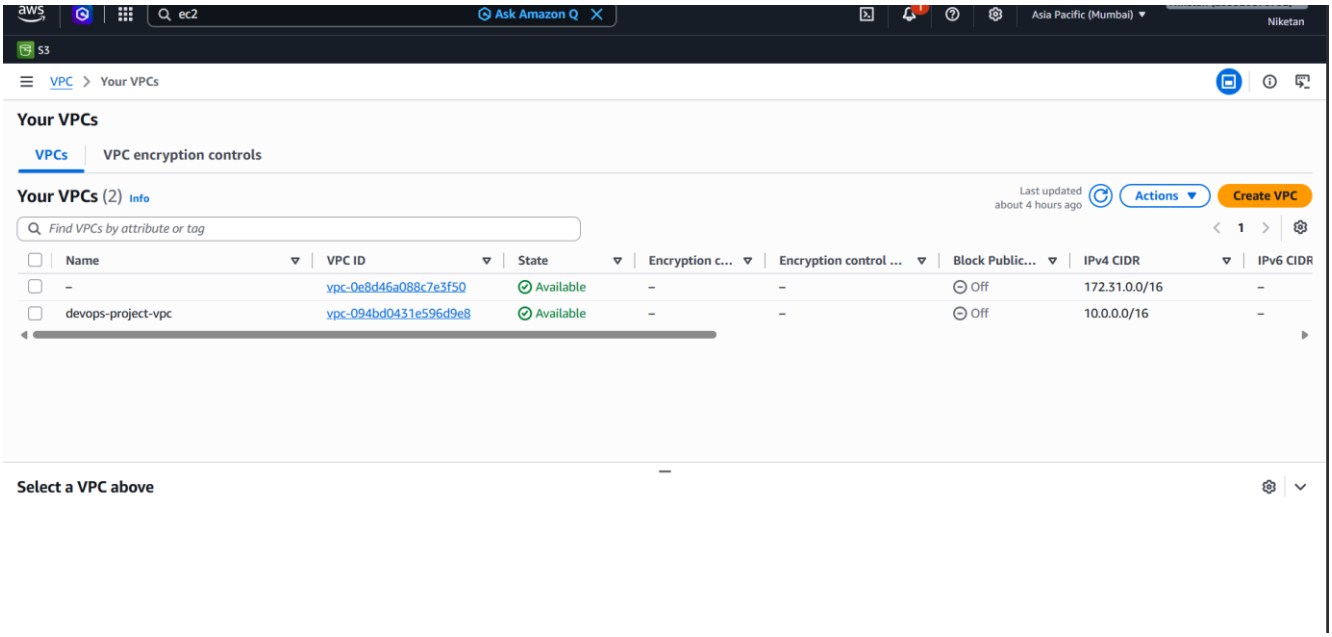


Figure: VPC details.

6.2 Create Four Subnets

Purpose: Separate public-facing components from private application servers and place resources across two Availability Zones.

Subnet	CIDR	Intended role
public-subnet-az1	10.0.1.0/24	Public subnet in AZ1
public-subnet-az2	10.0.2.0/24	Public subnet in AZ2
private-subnet-az1	10.0.11.0/24	Private application subnet in AZ1
private-subnet-az2	10.0.12.0/24	Private application subnet in AZ2

6. Open VPC → Subnets and choose Create subnet.
7. Select devops-project-vpc.
8. Create each subnet using the listed name, CIDR, and intended Availability Zone.
9. Verify that all four subnets exist in the correct VPC.

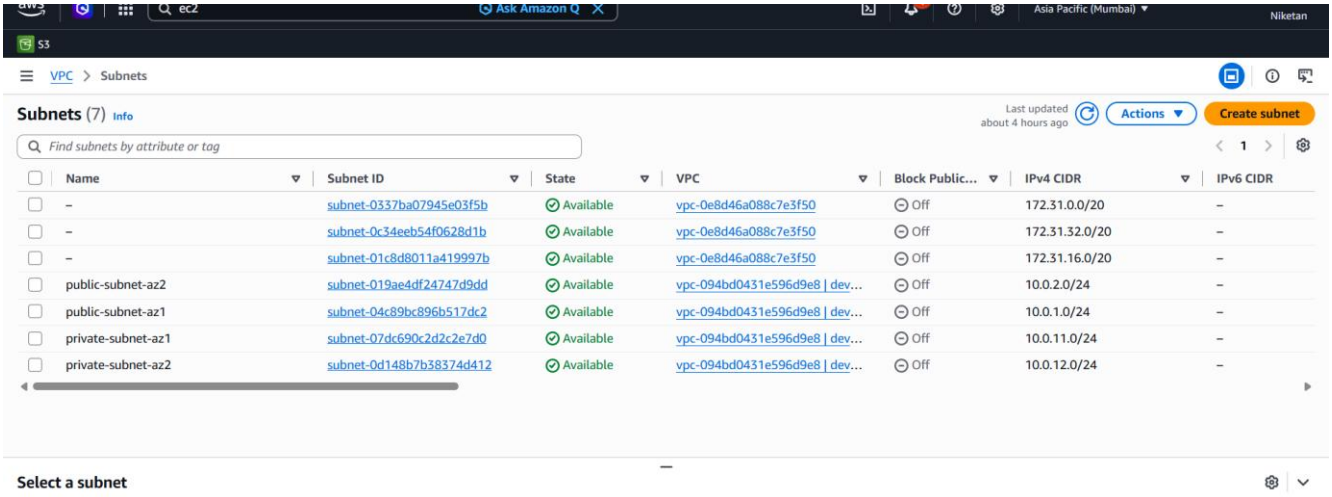


Figure: Subnet list.

6.3 Create and Attach the Internet Gateway

Purpose: Provide a route target for internet-bound traffic from public subnets.

10. Open VPC → Internet Gateways and choose Create internet gateway.
11. Name it devops-project-igw.
12. Select the new gateway and choose Actions → Attach to a VPC.
13. Attach it to devops-project-vpc and verify that its state is Attached.

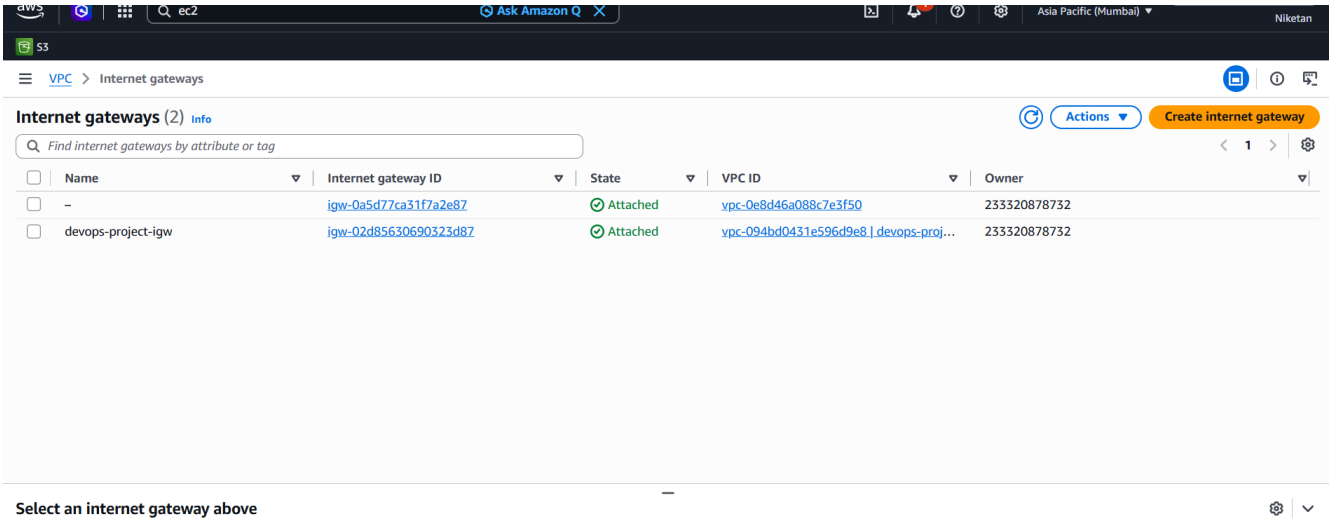


Figure: Internet Gateway attached.

6.4 Configure the Public Route Table

Purpose: Route internet-bound IPv4 traffic from public subnets through the Internet Gateway.

14. Create public-route-table in devops-project-vpc.
15. Under Routes, add 0.0.0.0/0 with target devops-project-igw.
16. Associate public-subnet-az1 and public-subnet-az2 with this route table.
17. Verify the local VPC route remains present.

Destination	Target	Purpose
10.0.0.0/16	local	Traffic within the VPC
0.0.0.0/0	Internet Gateway	Default IPv4 route for public subnets

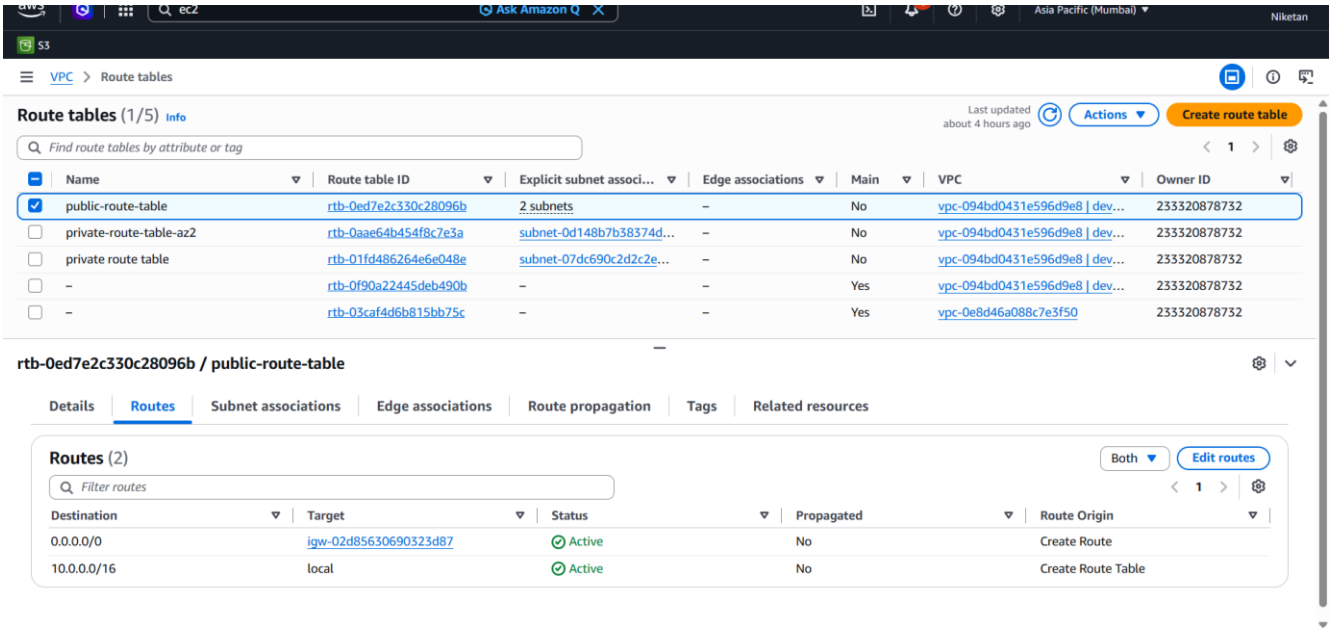


Figure: Public route table routes.

6.5 Configure Private Route Tables

Purpose: Keep application instances in private subnets and route general outbound internet traffic through the corresponding zonal NAT Gateway.

18. Use private-route-table for private-subnet-az1.
19. Create private-route-table-az2 for private-subnet-az2.
20. Associate only the matching private subnet with each route table.
21. Add 0.0.0.0/0 to nat-gateway-az1 in private-route-table.
22. Add 0.0.0.0/0 to nat-gateway-az2 in private-route-table-az2.
23. Verify that each table retains its local VPC route.

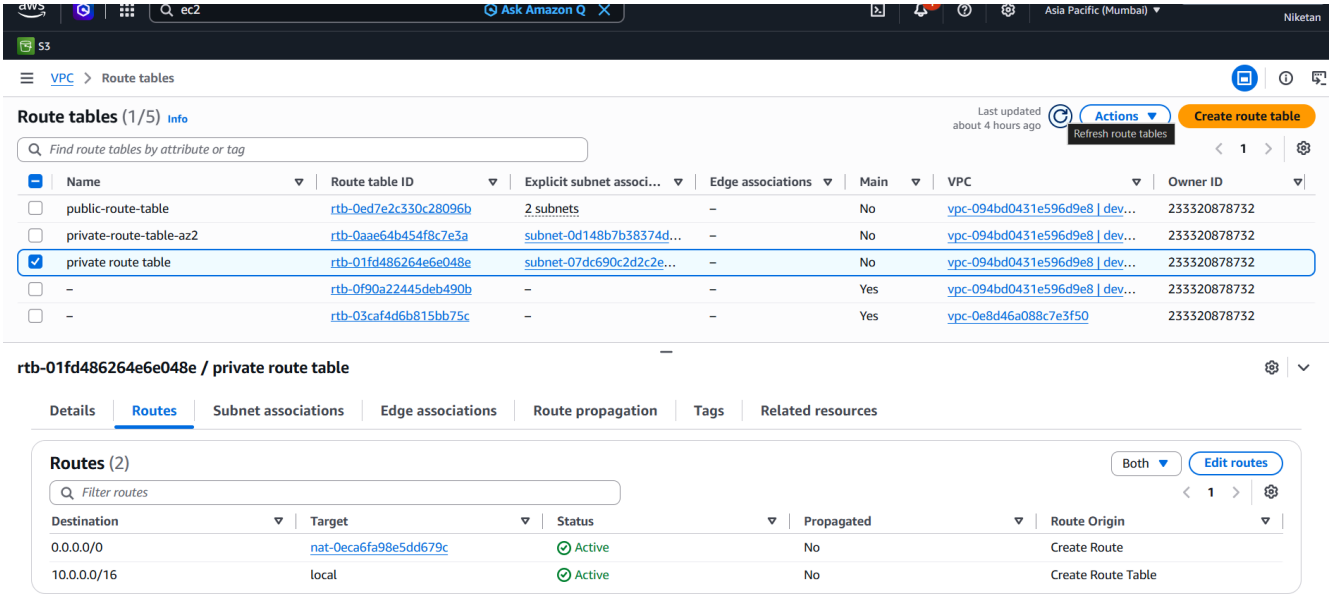


Figure: Private route table AZ1.

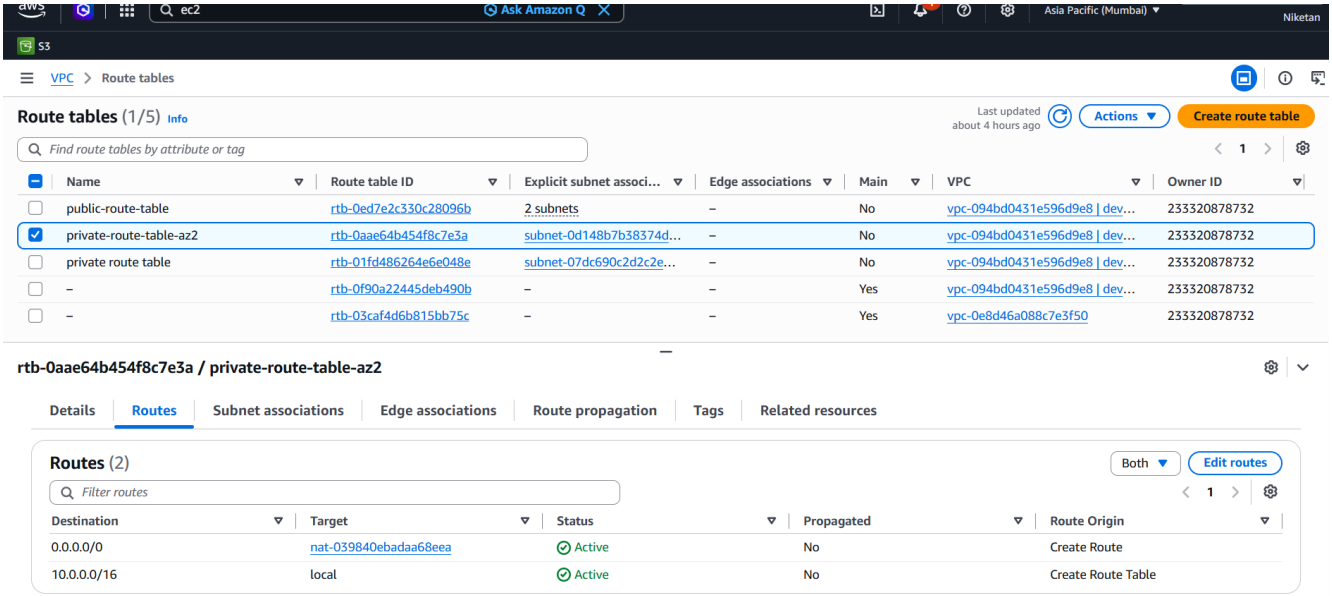


Figure: Private route table AZ2.

6.6 Create NAT Gateways

Purpose: Allow private instances to initiate outbound internet connections. Each public NAT Gateway uses an Elastic IP and is placed in a public subnet.

NAT Gateway	Mode	Subnet	Connectivity
nat-gateway-az1	Zonal	public-subnet-az1	Public
nat-gateway-az2	Zonal	public-subnet-az2	Public

24. Open VPC → NAT Gateways and choose Create NAT gateway.

25. Choose Zonal mode, devops-project-vpc, Public connectivity, and the matching public subnet.
26. Allocate an Elastic IP for each NAT Gateway.
27. Create both gateways and wait until their states are Available.
28. Confirm that each private route table points to its corresponding NAT Gateway.

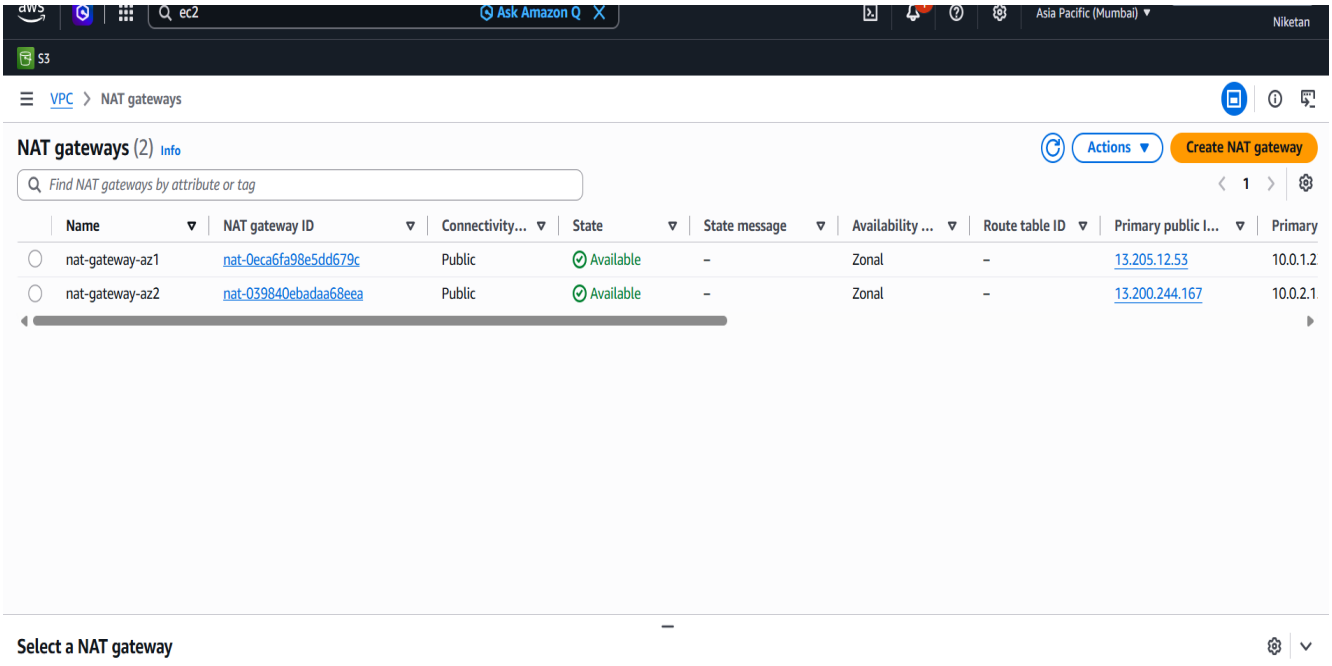


Figure: Both NAT Gateways.

6.7 Configure Security Groups

Purpose: Permit public HTTP traffic to the ALB while limiting EC2 HTTP traffic to the ALB Security Group.

Security Group	Inbound rule	Source
alb-security-group	HTTP / TCP / 80	0.0.0.0/0
ec2-security-group	HTTP / TCP / 80	alb-security-group

The default outbound rules were retained during setup. The EC2 Security Group should not allow HTTP from 0.0.0.0/0. The ALB Security Group is attached to the ALB; the EC2 Security Group is attached to the application instances.

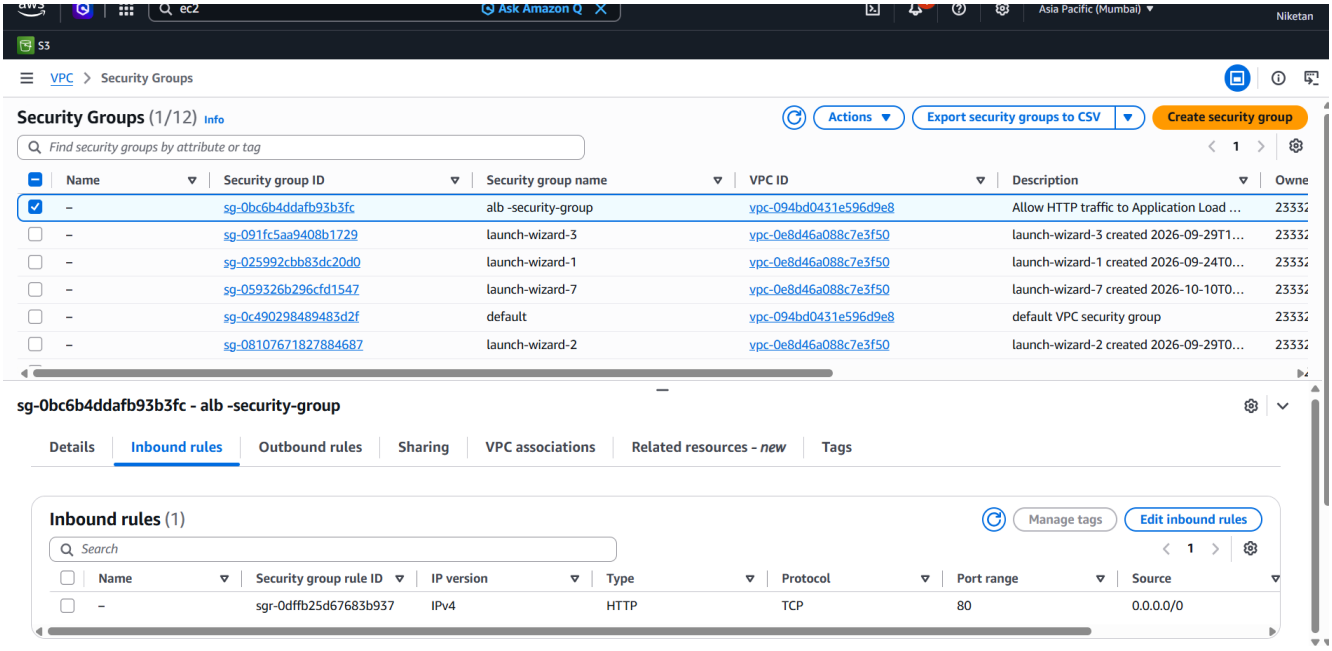


Figure: ALB Security Group.

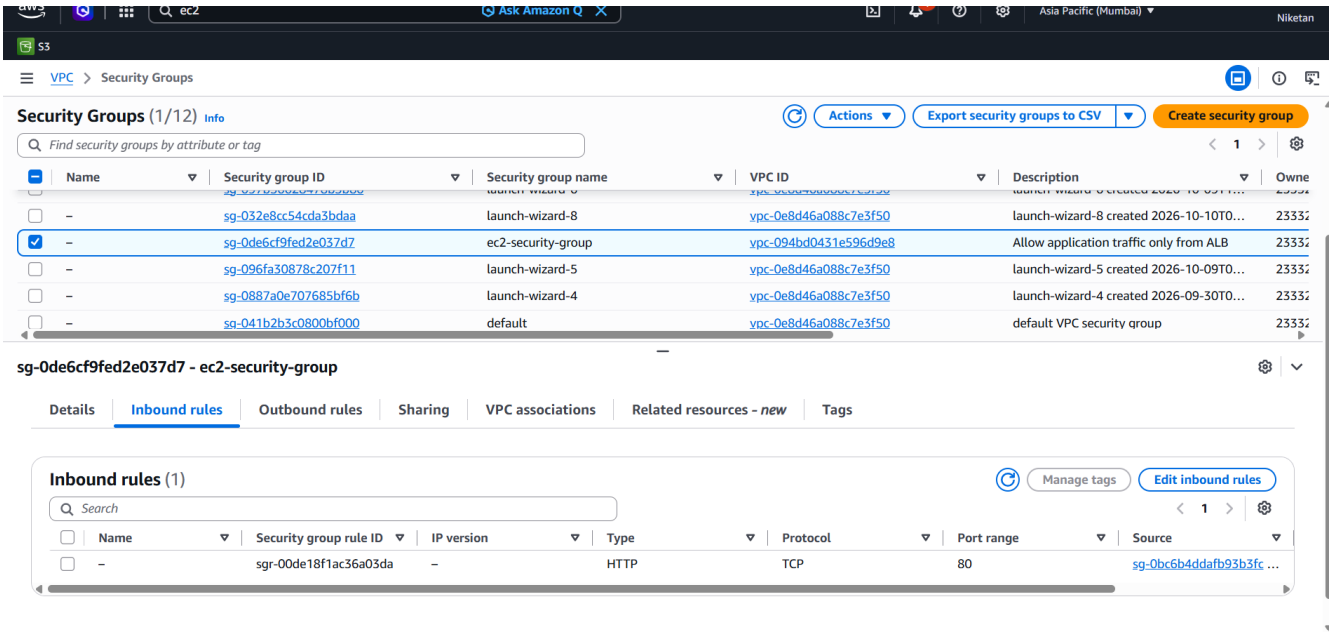


Figure: EC2 Security Group.

6.8 Create the S3 Bucket

Purpose: Provide object storage for project or application files. Record the exact bucket name from the AWS account.

29. Open Amazon S3 and choose Create bucket.
30. Select the Mumbai Region (ap-south-1) and General purpose bucket.
31. Enter a globally unique bucket name.
32. Keep Block Public Access enabled and use the recommended ACL-disabled ownership setting.
33. Keep default encryption enabled and create the bucket.

Actual bucket name: devops-project-file-nik

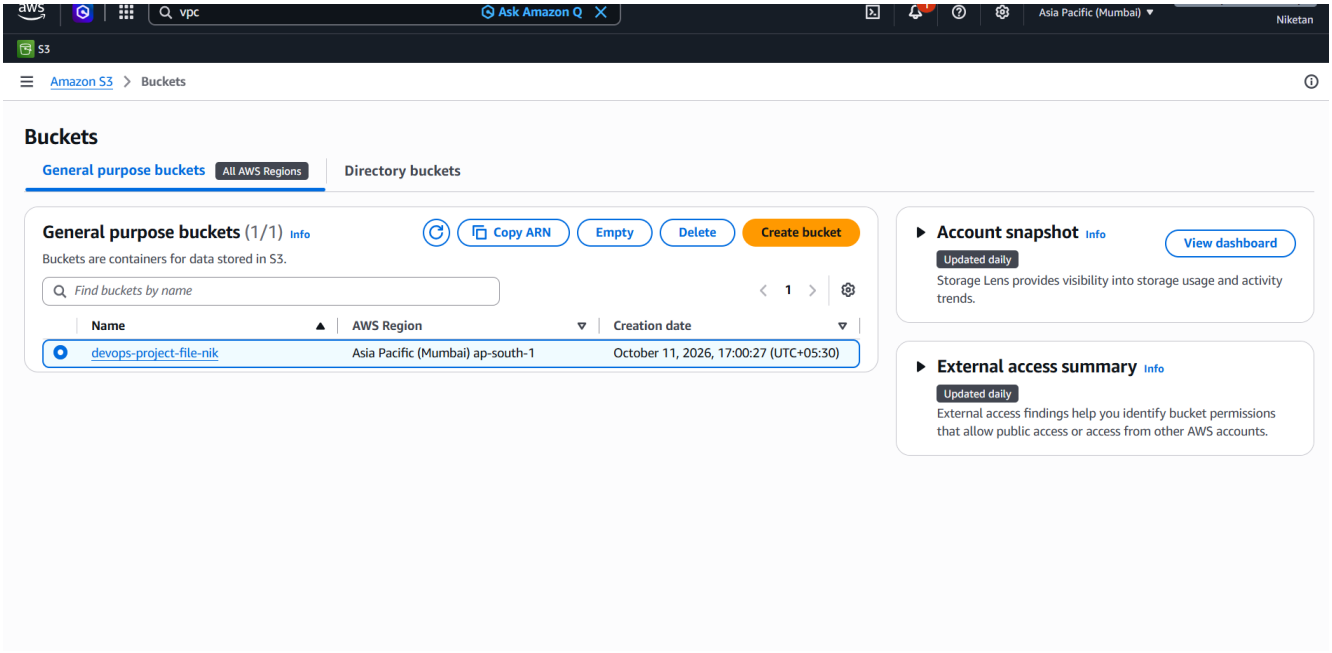


Figure: S3 bucket details.

6.9 Create the S3 Gateway Endpoint

Purpose: Route S3 traffic through a VPC Gateway Endpoint rather than through a NAT Gateway.

34. Open VPC → Endpoints and choose Create endpoint.
35. Choose AWS services and select the S3 Gateway service for ap-south-1: com.amazonaws.ap-south-1.s3.
36. Select devops-project-vpc and choose Gateway endpoint type.
37. Associate private-route-table and private-route-table-az2.
38. Keep the selected endpoint policy appropriate for the project; the default Full access policy was used during setup.
39. Create the endpoint and verify its state is Available.

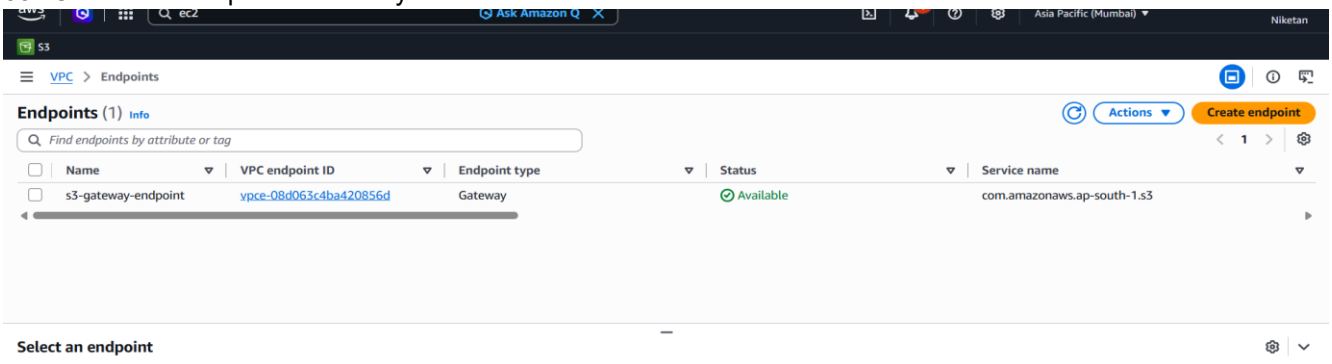


Figure: S3 Gateway Endpoint details.

6.10 Create the EC2 Launch Template

Purpose: Define a repeatable configuration used by the Auto Scaling Group to launch EC2 instances.

Parameter	Configured value
Launch Template	devops-webserver-template
AMI	Ubuntu Server LTS; record exact version from console if needed
Instance type	t3.micro
Subnet	Not fixed in template; selected by Auto Scaling Group
Security Group	ec2-security-group
Root volume	8 GiB gp3 as configured
User Data	Install Nginx, enable/start it, and write a basic HTML page

User Data script:

```
#!/bin/bash
apt-get update -y
apt-get install -y nginx
systemctl enable nginx
systemctl start nginx

cat > /var/www/html/index.html <<'EOF'
<!DOCTYPE html>
<html>
<head>
    <title>AWS DevOps Project</title>
</head>
<body>
    <h1>Welcome to My AWS DevOps Project!</h1>
    <p>High Availability Web Application</p>
</body>
</html>
EOF
```

User Data normally runs during the initial instance launch. Nginx must start successfully for the HTTP health check to pass.

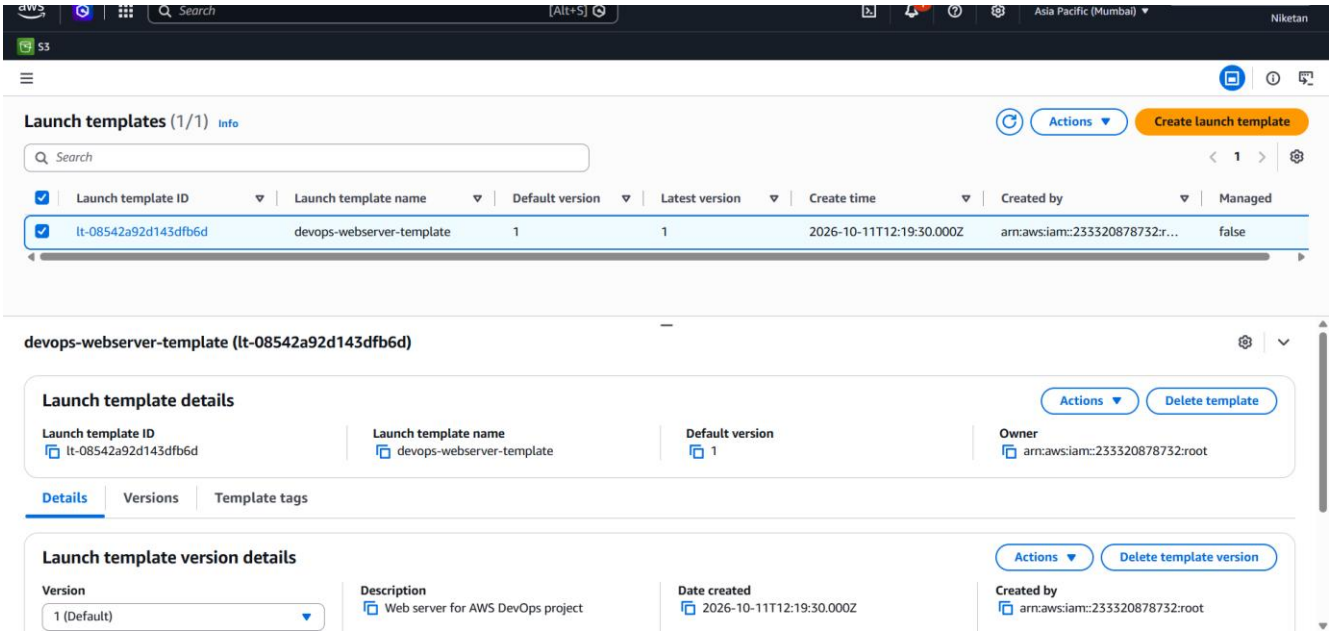


Figure: Launch Template details.

6.11 Create the Target Group

Purpose: Define backend EC2 targets and the health check used by the ALB.

Parameter	Value
Name	devops-webserver-tg
Target type	Instances
Protocol / port	HTTP / 80
Address type	IPv4
VPC	devops-project-vpc
Health check protocol	HTTP
Health check path	/

40. Open EC2 → Target Groups and choose Create target group.
41. Select Instances as the target type and enter devops-webserver-tg.
42. Set HTTP on port 80, select the project VPC, and set health check path to /.
43. Create the target group. The Auto Scaling Group registers instances when attached.

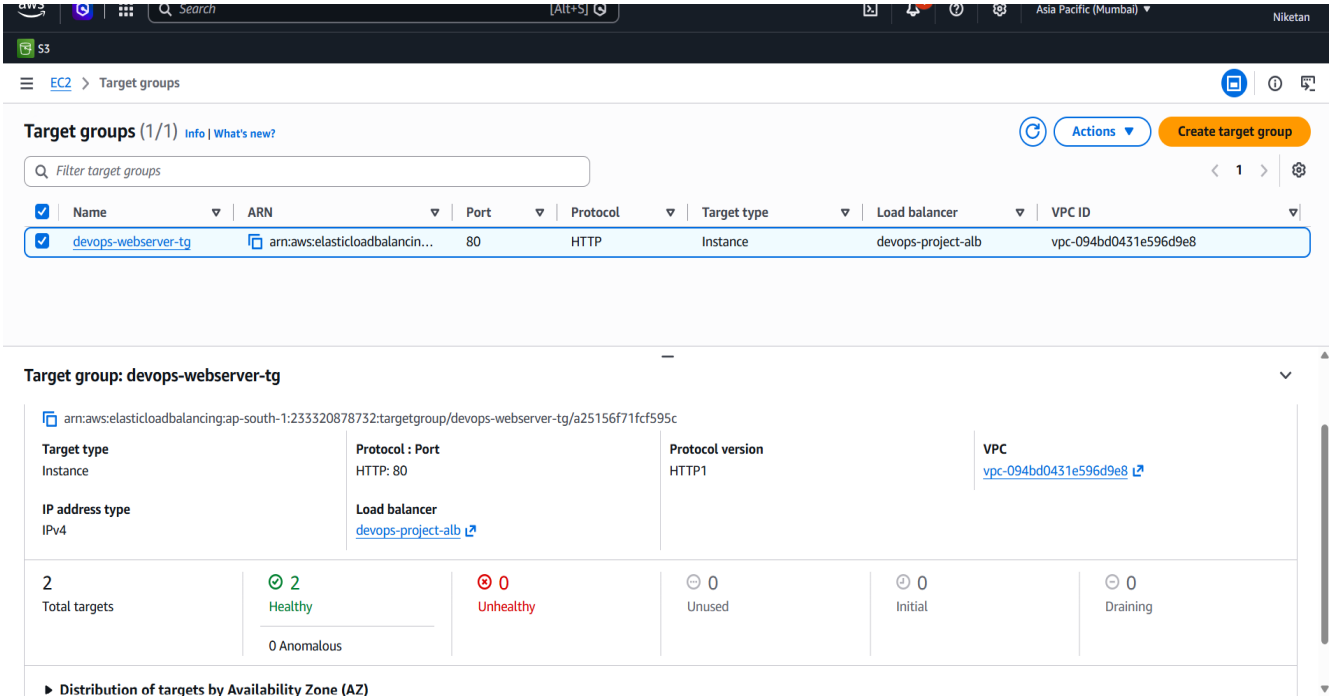


Figure: Target Group configuration.

6.12 Create the Application Load Balancer

Purpose: Receive client HTTP traffic and forward it to registered targets.

Parameter	Value
Name	devops-project-alb
Scheme	Internet-facing
IP address type	IPv4
VPC	devops-project-vpc
Subnets	public-subnet-az1 and public-subnet-az2
Security Group	alb-security-group
Listener	HTTP / 80
Default action	Forward to devops-webserver-tg

44. Open EC2 → Load Balancers and choose Create load balancer.

45. Select Application Load Balancer and enter devops-project-alb.
46. Choose Internet-facing and IPv4.
47. Select devops-project-vpc and both public subnets.
48. Attach alb-security-group.
49. Configure an HTTP listener on port 80 forwarding to devops-webserver-tg.
50. Create the ALB and wait for the state to become Active.

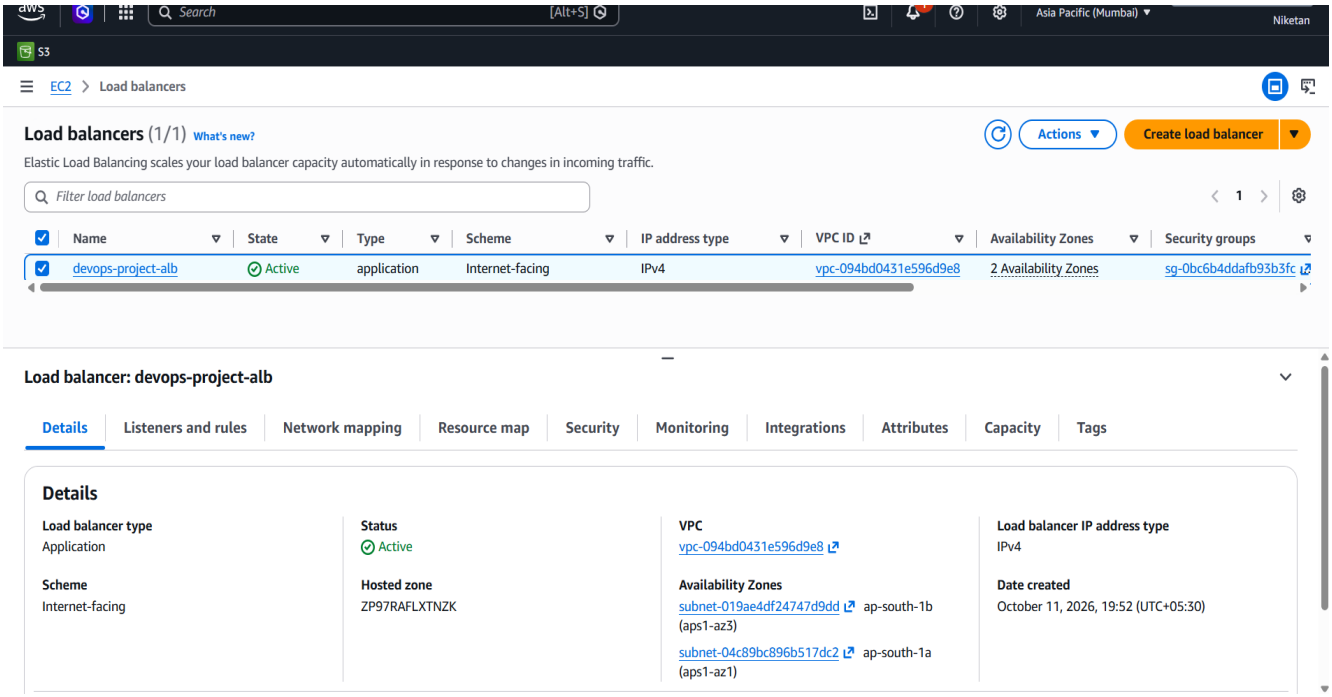


Figure: ALB details.

6.13 Create the Auto Scaling Group

Purpose: Launch and maintain EC2 instances across the two private subnets using the Launch Template.

Parameter	Configured value
ASG name	devops-project-asg
Launch Template	devops-webserver-template
VPC	devops-project-vpc
Subnets	private-subnet-az1 and private-subnet-az2
Target Group	devops-webserver-tg
Desired capacity	2
Minimum capacity	2
Maximum capacity	4

51. Open EC2 → Auto Scaling Groups and choose Create Auto Scaling group.
52. Select devops-webserver-template and enter devops-project-asg.
53. Select the project VPC and both private subnets.
54. Attach devops-webserver-tg.
55. Set desired capacity to 2, minimum capacity to 2, and maximum capacity to 4.
56. Review and create the Auto Scaling Group.
57. Verify the instance count and target registration.

Important: A maximum capacity of four does not automatically cause scale-out. An appropriate scaling policy or manual capacity change is required.

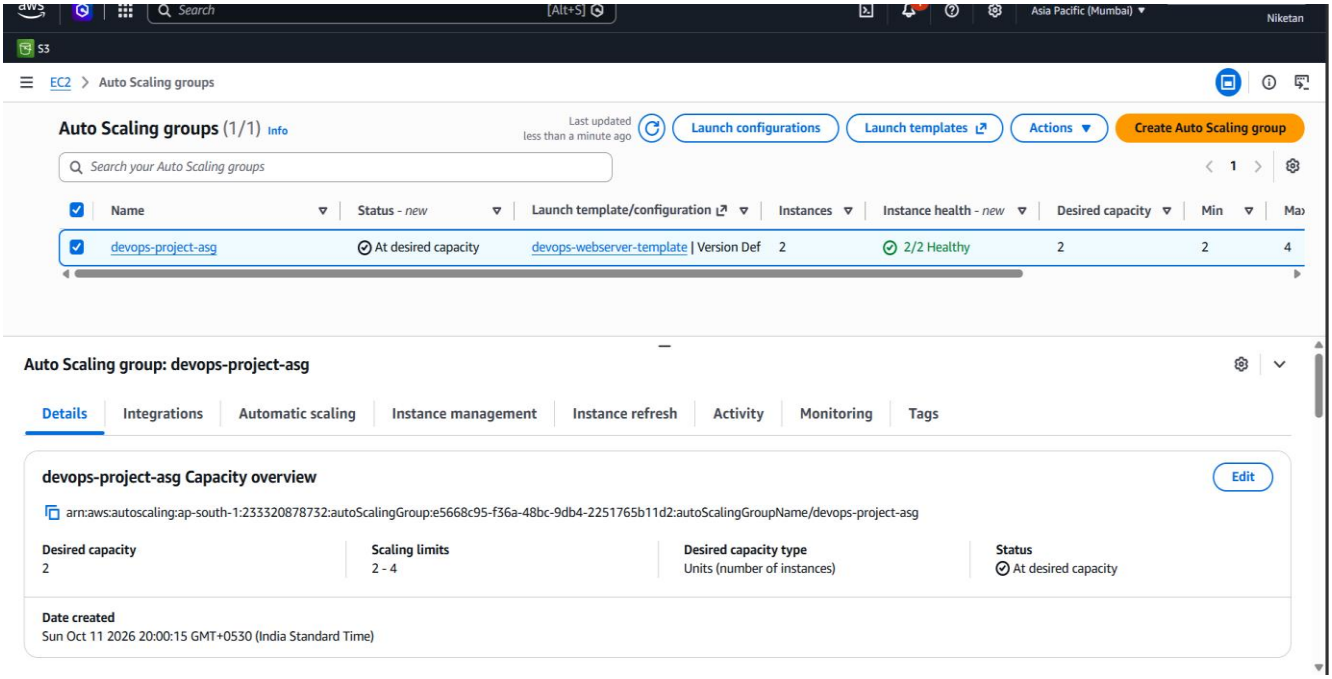


Figure: Auto Scaling Group settings.

7. Application Request Flow

58. A user opens the application using the DNS name of the internet-facing ALB.
59. The request reaches the ALB through the Internet Gateway and public subnet routing.
60. The ALB listener accepts the HTTP request on port 80.
61. The ALB forwards the request to devops-webserver-tg.
62. The target group routes the request to a healthy private EC2 instance running Nginx.
63. Nginx returns the HTML page, and the response travels through the ALB to the browser.

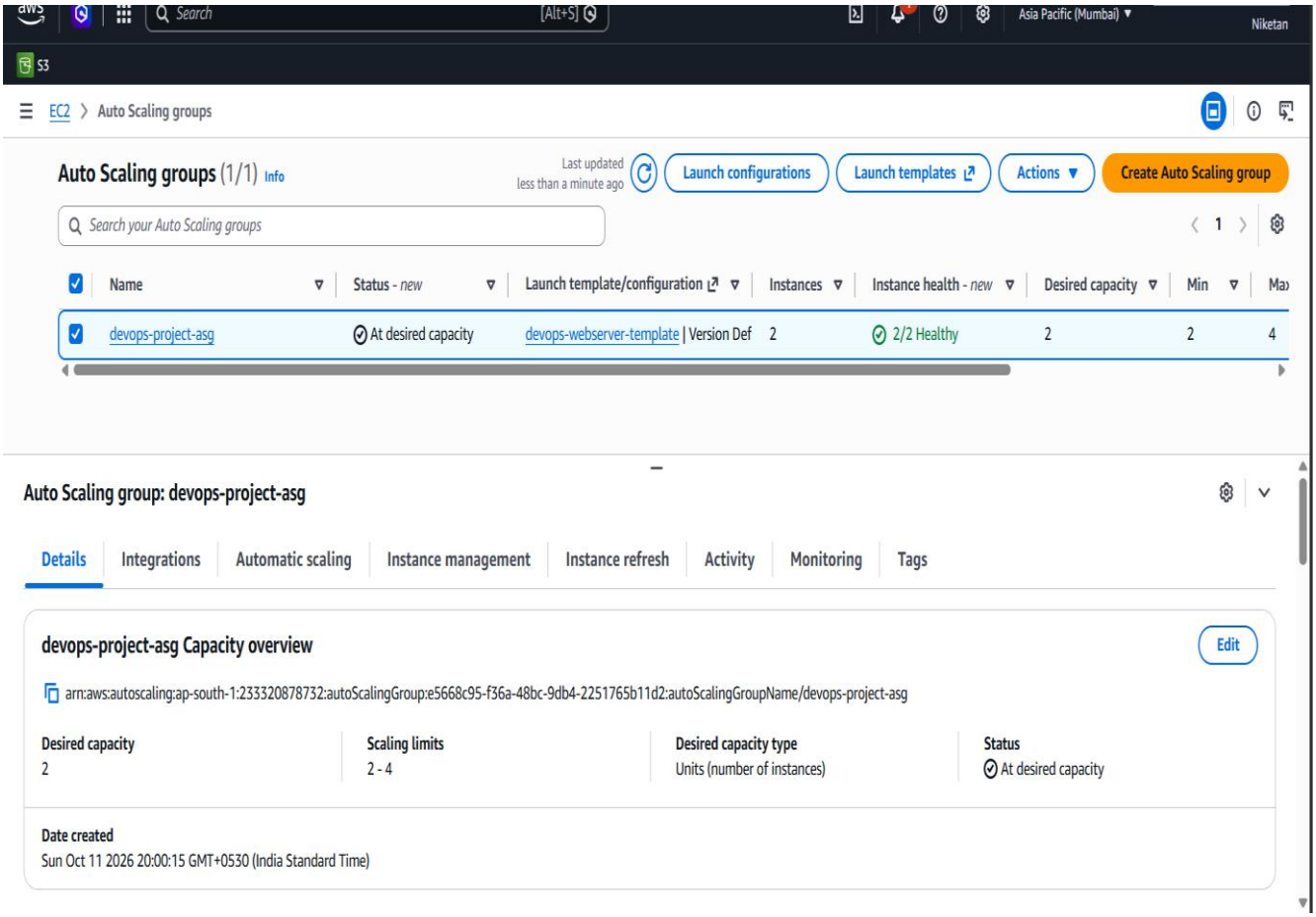


Figure: Application output.

8. Validation and Testing

Test	Where to verify	Record
VPC and subnets	VPC → Your VPCs / Subnets	Actual state, CIDRs, and AZs
Public routing	Public Route Table → Routes	Default route to Internet Gateway
Private routing	Private Route Tables → Routes	Routes to matching NAT Gateways and S3 endpoint association
NAT Gateways	VPC → NAT Gateways	Both were reported Available during setup
Target health	EC2 → Target Groups → Targets	Record actual target health state
ALB state	EC2 → Load Balancers	Record actual state
ASG capacity	EC2 → Auto Scaling Groups	Record configured capacities and actual instance count
Application response	Open ALB DNS in browser using http://	User reported that the application output appeared

Testing note: Automatic scale-out under load and recovery from an Availability Zone failure were not confirmed as completed tests. Do not mark them as passed unless performed and documented.



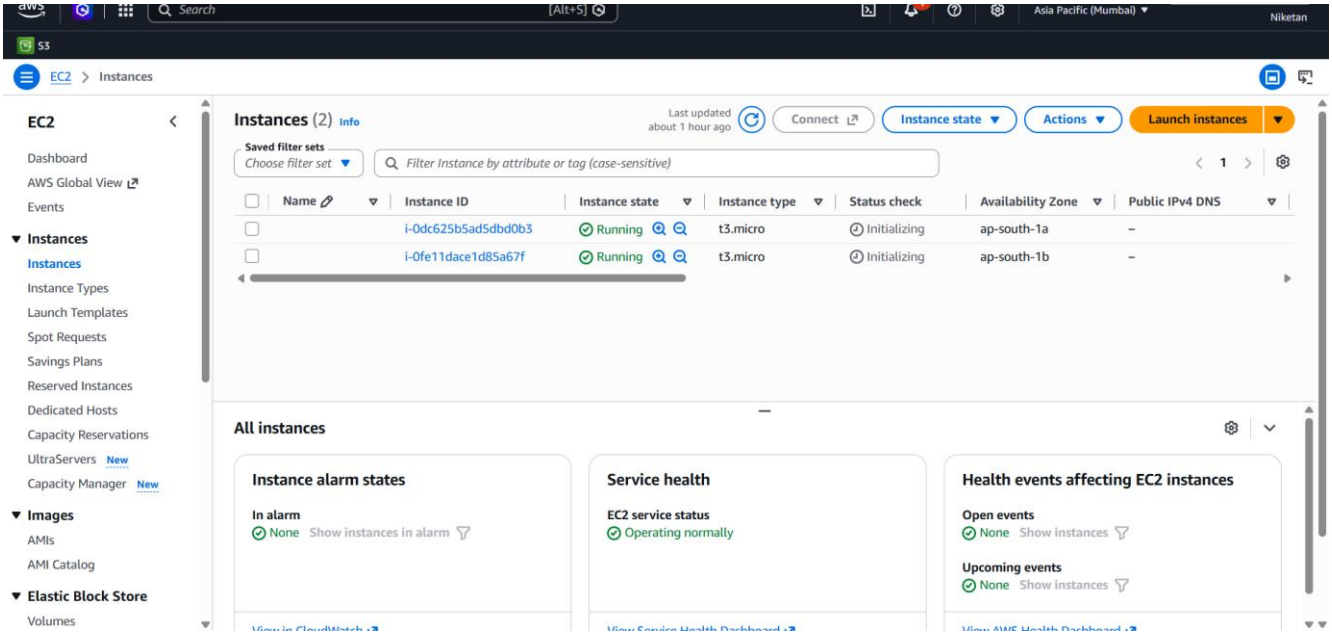


Figure: Final test results.

9. Security Considerations

- EC2 application instances are placed in private subnets and are not intended to be directly reachable from the public internet.
- The ALB Security Group allows HTTP port 80 from 0.0.0.0/0 for this demonstration.
- The EC2 Security Group allows HTTP port 80 only from alb-security-group.
- Keep S3 Block Public Access enabled unless a reviewed requirement needs otherwise.
- Never commit AWS access keys, secret keys, passwords, private keys, .pem files, or sensitive account information to GitHub.
- HTTP is used for the demonstration; HTTPS/TLS is a potential future improvement.

10. Cost and Cleanup Considerations

Resources in this project can incur charges, especially NAT Gateways, the Application Load Balancer, EC2 instances, Elastic IP/public IPv4 addresses, and S3 storage or requests. Charges depend on usage, Region, and current AWS pricing. Delete resources when they are no longer needed.

Recommended cleanup order: delete the Auto Scaling Group and verify its instances terminate; delete the ALB; delete the Target Group and Launch Template; delete the S3 Gateway Endpoint and S3 bucket if its contents are no longer needed; delete NAT Gateways and release unused Elastic IPs; delete custom Security Groups and Route Tables; detach and delete the Internet Gateway; delete the four subnets; finally delete the custom VPC.

Cleanup status must be updated based on the actual AWS Console. This document does not assume cleanup has been completed.

11. Future Improvements

- Configure HTTPS with a TLS certificate and HTTPS listener.
- Add a custom domain and DNS configuration if required.
- Configure and test a target-tracking or step-scaling policy.
- Test instance replacement and document the outcome.
- Add monitoring and alerting with Amazon CloudWatch.
- Recreate the infrastructure with Terraform or AWS CloudFormation.
- Add a CI/CD pipeline for automated deployment.
- Apply least-privilege IAM roles and an S3 bucket policy tailored to the application.

12. Conclusion

This project provided practical experience with AWS networking, public/private subnet design, route tables, Internet and NAT Gateways, Security Groups, EC2 provisioning, Application Load Balancing, Auto Scaling configuration, and S3 Gateway Endpoint connectivity. A basic Nginx web page was deployed on EC2 instances and accessed through the ALB DNS name. The project demonstrates how AWS network and compute services can be combined to build a web application architecture designed for availability and horizontal scaling.

Advanced behavior such as automatic scale-out under load or failure recovery should be described as completed only after testing.



Figure: Project completion evidence.

Project Created & Documented By
Niketan Sarode.